Project 3a

This is the first part of the project. We will be exploring different kinds of methods to apply to our dataset.

Submission Details

Project is due June 13th at 11:59 pm (Friday Midnight). To submit the project, please save the notebook as a pdf file and submit the assignment via Gradescope. In addition, make sure that all figures are legible and sufficiently large. For best pdf results, we recommend printing the notebook using $\LaTeX$

Loading Essentials and Helper Functions

In [1]:
```
# fix for windows memory leak with MKL
import os
import platform

if platform.system() == "Windows":
    os.environ["OMP_NUM_THREADS"] = "2"
```

In [2]:
```
# import libraries
import time
import random
import numpy as np  # linear algebra
import pandas as pd  # data processing, CSV file I/O (e.g. pd.read_csv)
import matplotlib.pyplot as plt  # this is used for the plot the graph

# Sklearn classes
from sklearn.model_selection import (
    train_test_split,
    cross_val_score,
    GridSearchCV,
    KFold,
)
from sklearn import metrics
from sklearn.metrics import confusion_matrix, silhouette_score
import sklearn.metrics.cluster as smc
from sklearn.cluster import KMeans
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.pipeline import Pipeline, FeatureUnion
from sklearn.preprocessing import (
    StandardScaler,
    OneHotEncoder,
    LabelEncoder,
    MinMaxScaler,
```

```
)
from sklearn.compose import ColumnTransformer, make_column_transformer
from sklearn import tree
from sklearn import datasets
from sklearn.decomposition import PCA
from sklearn.neural_network import MLPClassifier
from sklearn.datasets import make_blobs

from helper import (
    draw_confusion_matrix,
    heatmap,
    make_meshgrid,
    plot_contours,
    draw_contour,
)

%matplotlib inline
%config InlineBackend.figure_format = 'retina'

# Sets random seed for reproducibility
SEED = 66
random.seed(SEED)
```

Background: Dataset Information (Recap)

For this exercise we will be using a subset of the UCI Heart Disease dataset, leveraging the fourteen most commonly used attributes. All identifying information about the patient has been scrubbed. You will be asked to classify whether a **patient is suffering from heart disease** based on a host of potential medical factors.

The dataset includes 14 columns. The information provided by each column is as follows:

- **age:** Age in years
- **sex:** (male/female)
- **cp:** Chest pain type (0 = asymptomatic; 1 = atypical angina; 2 = non-anginal pain; 3 = typical angina)
- **trestbps:** Resting blood pressure (in mm Hg on admission to the hospital)
- **chol:** cholesterol in mg/dl
- **fbs** Fasting blood sugar > 120 mg/dl (1 = true; 0 = false)
- **restecg:** Resting electrocardiographic results (0= showing probable or definite left ventricular hypertrophy by Estes' criteria; 1 = normal; 2 = having ST-T wave abnormality (T wave inversions and/or ST elevation or depression of > 0.05 mV))
- **thalach:** Maximum heart rate achieved
- **exang:** Exercise induced angina (1 = yes; 0 = no)
- **oldpeak:** Depression induced by exercise relative to rest

- **slope:** The slope of the peak exercise ST segment (0 = downsloping; 1 = flat; 2 = upsloping)
- **ca:** Number of major vessels (0-3) colored by flourosopy
- **thal:** 1 = normal; 2 = fixed defect; 7 = reversable defect
- **sick:** Indicates the presence of Heart disease (True = Disease; False = No disease)

Preprocess Data

This part is done for you since you would have already completed it in project 2. Use the train, target, test, and target_test for all future parts. We also provide the column names for each transformed column for future use.

In [11]:
```
# Preprocess Data

# Load Data
data = pd.read_csv("datasets/heartdisease.csv")

# Transform target feature into numerical
le = LabelEncoder()
data["target"] = le.fit_transform(data["sick"])
data["sex"] = le.fit_transform(data["sex"])
data = data.drop(["sick"], axis=1)

# Split target and data
y = data["target"]
x = data.drop(["target"], axis=1)

# Train test split
# 35% in test data
train_raw, test_raw, target, target_test = train_test_split(
    x, y, test_size=0.35, stratify=y, random_state=0
)

# Feature Transformation
# This is the only change from project 2 since we replaced standard scaler t
# This was done to ensure that the numerical features were still of the same
# as the one hot encoded features
num_pipeline = Pipeline([("minmax", MinMaxScaler())])

heart_num = train_raw.drop(
    ["sex", "cp", "fbs", "restecg", "exang", "slope", "ca", "thal"], axis=1
)
numerical_features = list(heart_num)
categorical_features = ["sex", "cp", "fbs", "restecg", "exang", "slope", "ca

full_pipeline = ColumnTransformer(
    [
        ("num", num_pipeline, numerical_features),
        ("cat", OneHotEncoder(categories="auto"), categorical_features),
    ]
```

```
)

# Transform raw data|
train = full_pipeline.fit_transform(train_raw)
test = full_pipeline.transform(test_raw)  # Note that there is no fit calls

# Extracts features names for each transformed column
feature_names = full_pipeline.get_feature_names_out(list(x.columns))
```

In [12]:
```
print("Column names after transformation by pipeline: ", feature_names)
```

```
Column names after transformation by pipeline:  ['num__age' 'num__trestbps'
'num__chol' 'num__thalach' 'num__oldpeak'
 'cat__sex_0' 'cat__sex_1' 'cat__cp_0' 'cat__cp_1' 'cat__cp_2' 'cat__cp_3'
 'cat__fbs_0' 'cat__fbs_1' 'cat__restecg_0' 'cat__restecg_1'
 'cat__restecg_2' 'cat__exang_0' 'cat__exang_1' 'cat__slope_0'
 'cat__slope_1' 'cat__slope_2' 'cat__ca_0' 'cat__ca_1' 'cat__ca_2'
 'cat__ca_3' 'cat__ca_4' 'cat__thal_0' 'cat__thal_1' 'cat__thal_2'
 'cat__thal_3']
```

The following shows the baseline accuracy of simply classifying every sample as the majority class.

In [5]:
```
# Baseline accuracy of using the majority class
ct = target_test.value_counts()
print("Counts of each class in target_test: ")
print(ct)
print(
    "==========================================",
    "\nBaseline Accuracy of using Majority Class:",
    np.round(np.max(ct) / np.sum(ct), 3),
)
```

```
Counts of each class in target_test:
target
0    58
1    49
Name: count, dtype: int64
==========================================
Baseline Accuracy of using Majority Class: 0.542
```

1. (15 pts) Decision Trees

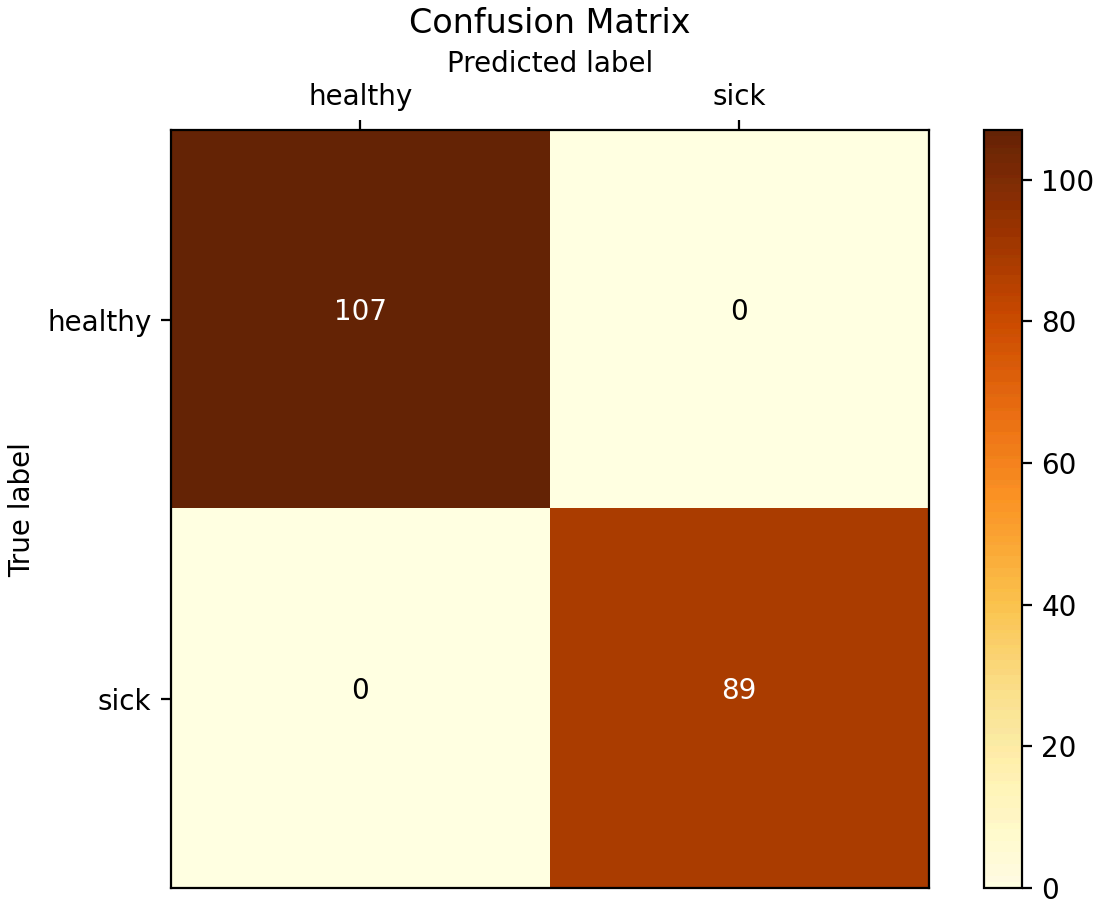
1.1. [2.5 pts] Apply Decision Tree on Train Data

Apply the decision tree on the **train data** with default parameters of the DecisionTreeClassifier. **Report the accuracy and print the confusion matrix**. Make sure to use `random_state = SEED` so that your results match ours.

In [10]:
```
from sklearn import tree
clf = DecisionTreeClassifier(random_state=SEED)
clf.fit(train, target)
predicted = clf.predict(train)
```

```
print("%-12s %f" % ("Accuracy:", metrics.accuracy_score(target, predicted)))
print("Confusion Matrix: \n", metrics.confusion_matrix(target, predicted))
draw_confusion_matrix(target, predicted, ["healthy", "sick"])
```

```
Accuracy:    1.000000
Confusion Matrix:
 [[107   0]
 [  0  89]]
```

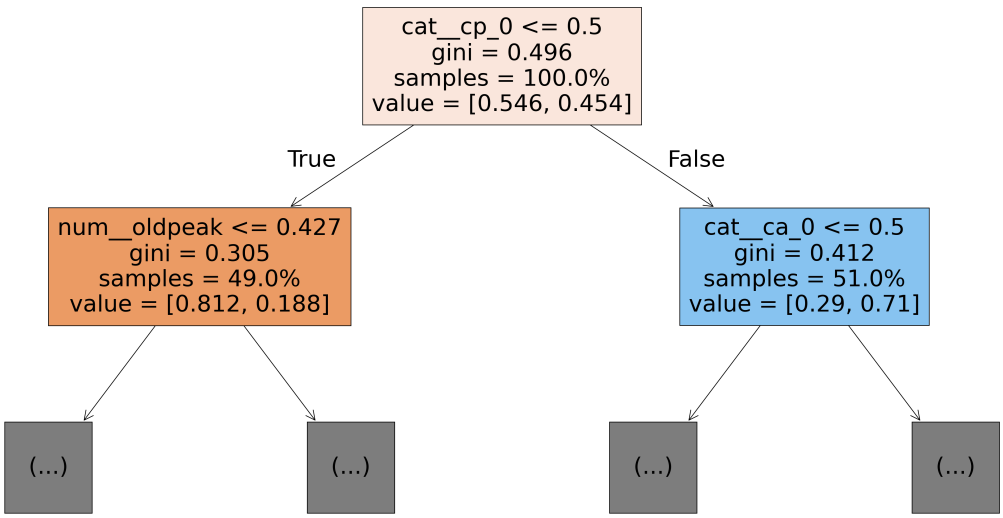


1.2. [2.5 pts] Visualize the Decision Tree

Visualize the first two layers of the decision tree that you trained.

In [13]:
```
plt.figure(figsize=(30, 15))
tree.plot_tree(clf, max_depth=1, proportion=True, feature_names=feature_name
```

Out[13]:
```
[Text(0.5, 0.8333333333333334, 'cat__cp_0 <= 0.5\ngini = 0.496\nsamples = 1
00.0%\nvalue = [0.546, 0.454]'),
 Text(0.25, 0.5, 'num__oldpeak <= 0.427\ngini = 0.305\nsamples = 49.0%\nval
ue = [0.812, 0.188]'),
 Text(0.375, 0.6666666666666667, 'True  '),
 Text(0.125, 0.16666666666666666, '\n  (...)  \n'),
 Text(0.375, 0.16666666666666666, '\n  (...)  \n'),
 Text(0.75, 0.5, 'cat__ca_0 <= 0.5\ngini = 0.412\nsamples = 51.0%\nvalue =
[0.29, 0.71]'),
 Text(0.625, 0.6666666666666667, '  False'),
 Text(0.625, 0.16666666666666666, '\n  (...)  \n'),
 Text(0.875, 0.16666666666666666, '\n  (...)  \n')]
```



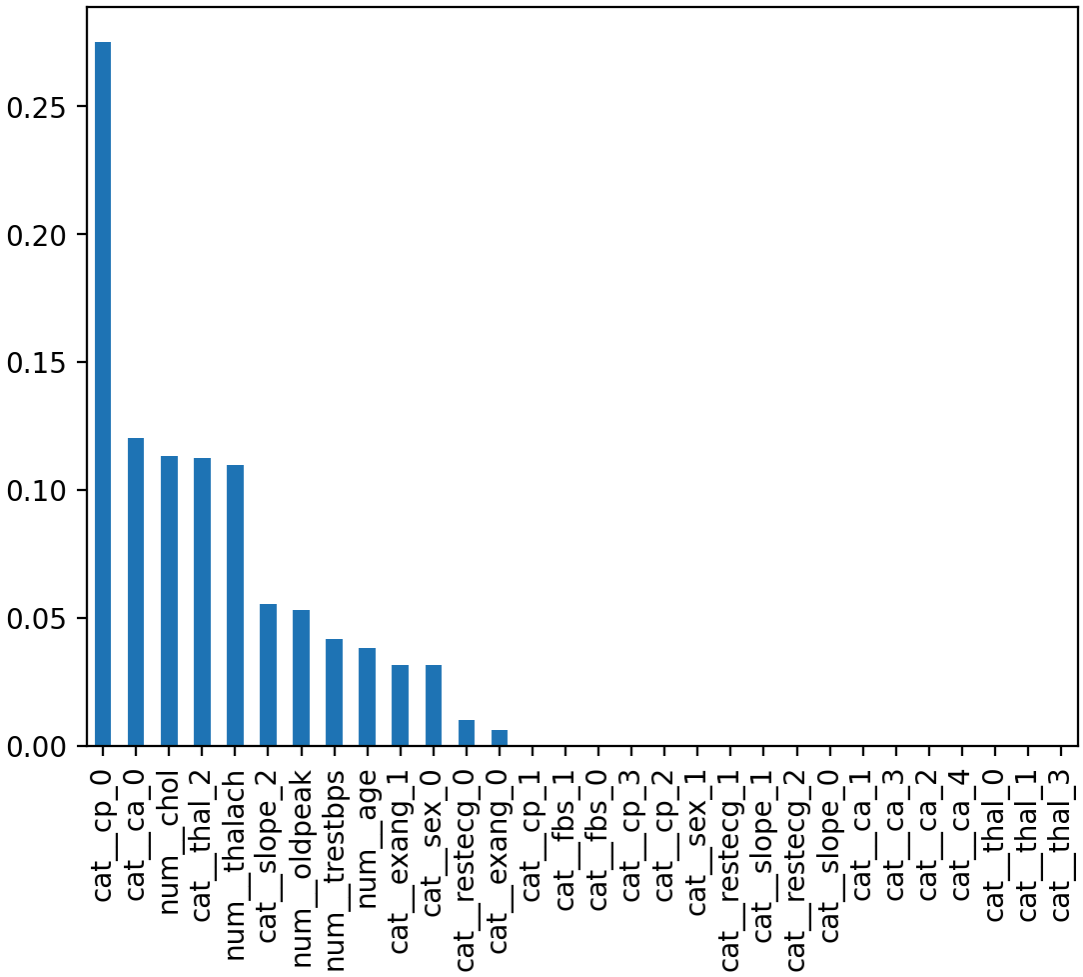
What is the gini index improvement of the first split? Give an interpretation of the result

Improvement = 0.496 - ((0.49 × 0.305) + (0.51 × 0.412)) = .137. As a result, the split gives us a great reduction in impurity which shows that chest pain type is an important feature. After the split, each node has a much clearer majority class and shows the split is effective that separating classes.

[2.5 pts] Plot the importance of each feature for the Decision Tree

In [14]:
```
imp_pd = pd.Series(data=clf.feature_importances_, index=feature_names)
imp_pd = imp_pd.sort_values(ascending=False)
imp_pd.plot.bar()
```

Out[14]:
```
<Axes: >
```



How many features have non-zero importance for the Decision Tree? If we remove the features with zero importance, will it change the decision tree for the same sampled dataset?

13 features have non-zero importance. Removing features with zero importance will not change the decision tree.

[7.5 pts] Optimize Decision Tree

While the default Decision Tree performs fairly well on the data, lets see if we can improve performance by optimizing the parameters.

Run a `GridSearchCV` with 5-Fold Cross Validation for the Decision Tree. Find the best model parameters for accuracy amongst the following:

- `max_depth` = [4, 8, 16, 32, 64]
- `min_samples_split` = [4, 8, 16, 32]
- `criterion` = [`gini` , `entropy` , `log_loss`]

After using `GridSearchCV` , Print the **best 10 models** with the following parameters: `rank_test_score` , `param_max_depth` ,

`param_min_samples_split`, `param_criterion`, `mean_test_score`, `std_test_score`.

In [20]:
```
from sklearn.model_selection import GridSearchCV
from sklearn.tree import DecisionTreeClassifier
params = {'max_depth': [4, 8, 16, 32, 64], 'min_samples_split': [4, 8, 16, 3
clf = DecisionTreeClassifier(random_state=SEED)
grid_search = GridSearchCV(clf, params, cv=5)
grid_search.fit(train, target)
print("Best parameters: ", grid_search.best_params_)
print("Best score: ", grid_search.best_score_)
results_df = pd.DataFrame(grid_search.cv_results_)
best_10 = results_df[['rank_test_score', 'param_max_depth', 'param_min_sampl
                      'param_criterion', 'mean_test_score', 'std_test_score']
print("\nBest 10 models:")
print(best_10.to_string(index=False))
```

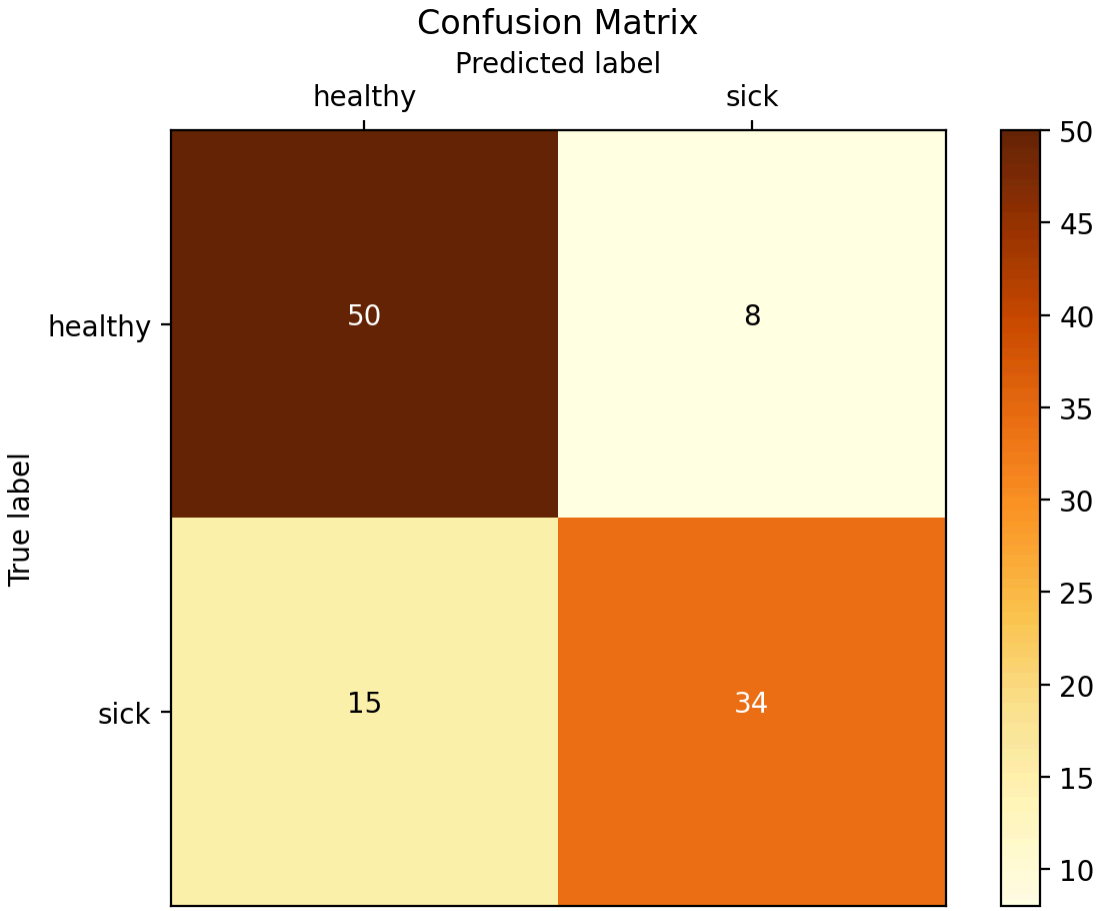
```
Best parameters:  {'criterion': 'entropy', 'max_depth': 4, 'min_samples_spli
t': 16}
Best score:  0.7547435897435897

Best 10 models:
 rank_test_score  param_max_depth  param_min_samples_split param_criterion
mean_test_score  std_test_score
              35                4                        4            gini
0.734231        0.069197
              35                4                        8            gini
0.734231        0.069197
              16                4                       16            gini
0.739359        0.060618
              11                4                       32            gini
0.739487        0.048260
              17                8                        4            gini
0.734615        0.035251
              57                8                        8            gini
0.719231        0.059086
              21                8                       16            gini
0.734359        0.054019
              11                8                       32            gini
0.739487        0.048260
              17               16                        4            gini
0.734615        0.035251
              57               16                        8            gini
0.719231        0.059086
```

Using the best model you have, report the test accuracy and print out the confusion matrix

In [19]:
```
best_model = grid_search.best_estimator_
test_predictions = best_model.predict(test)
test_accuracy = metrics.accuracy_score(target_test, test_predictions)
print("%-12s %f" % ("Accuracy:", metrics.accuracy_score(target_test, test_pr
print("Confusion Matrix: \n", metrics.confusion_matrix(target_test, test_pre
draw_confusion_matrix(target_test, test_predictions, ["healthy", "sick"])
```

```
Accuracy:    0.785047
Confusion Matrix:
 [[50  8]
 [15 34]]
```



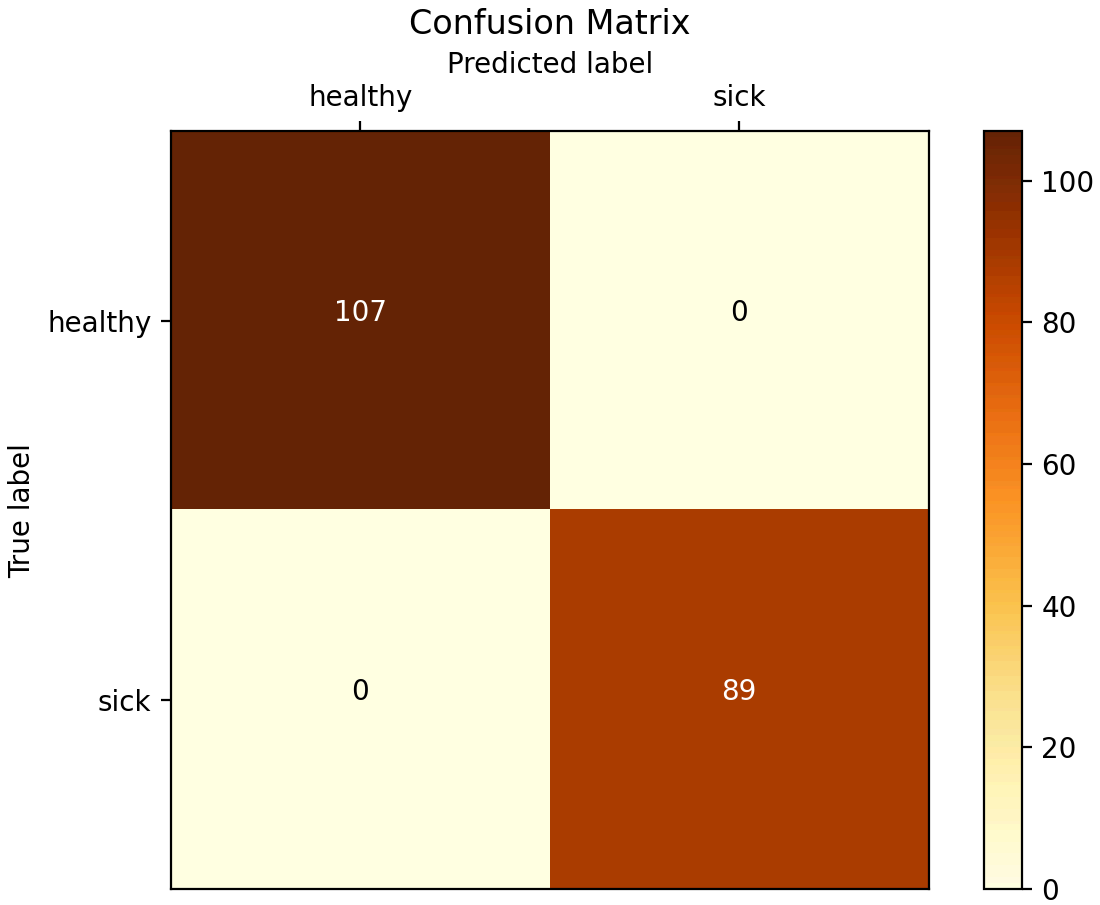
(10 pts) Multi-Layer Perceptron

[2.5 pts] Applying a Multi-Layer Perceptron

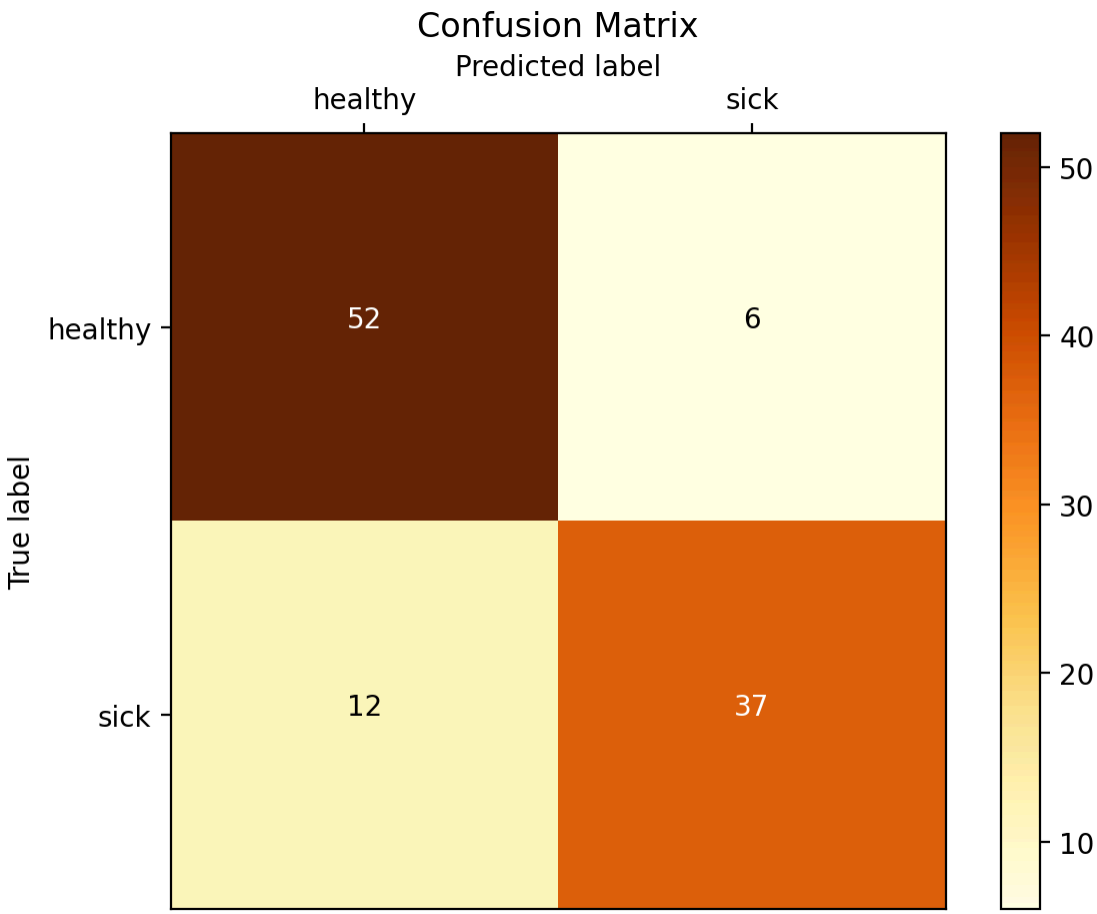
Apply the MLP on the **train data** with hidden_layer_sizes=(100, 100) and max_iter = 1000. **Report the accuracy and print the confusion matrix**. Make sure to set `random_state=SEED`.

In [34]:
```
mlp = MLPClassifier(hidden_layer_sizes=(100, 100), max_iter=1000, random_sta
mlp.fit(train, target)
predicted = mlp.predict(train)
train_accuracy = metrics.accuracy_score(target_test, test_predictions)
print("%-12s %f" % ("Accuracy:", metrics.accuracy_score(target, predicted)))
print("Confusion Matrix: \n", metrics.confusion_matrix(target, predicted))
draw_confusion_matrix(target, predicted, ["healthy", "sick"])
test_predictions = mlp.predict(test)
test_accuracy = metrics.accuracy_score(target_test, test_predictions)
print("%-12s %f" % ("Accuracy:", test_accuracy))
print("Confusion Matrix: \n", metrics.confusion_matrix(target_test, test_pre
draw_confusion_matrix(target_test, test_predictions, ["healthy", "sick"])
```

```
Accuracy:    1.000000
Confusion Matrix:
 [[107   0]
 [  0  89]]
```




```
Accuracy:    0.831776
Confusion Matrix:
 [[52  6]
 [12 37]]
```



[5 pts] Speedtest between Decision Tree and MLP

Let us compare the training times and prediction times of a Decision Tree and an MLP. **Time how long it takes for a Decision Tree and an MLP to perform a .fit operation (i.e. training the model). Then, time how long it takes for a Decision Tree and an MLP to perform a .predict operation (i.e. predicting the testing data). Print out the timings and specify which model was quicker for each operation.** We recommend using the time python module to time your code. An example of the time module was shown in project 2. Use the default Decision Tree Classifier and the MLP with the previously mentioned parameters.

In [24]:
```
import time
dt = DecisionTreeClassifier(random_state=SEED)
start = time.time()
dt.fit(train, target)
dt_train_time = time.time() - start
start = time.time()
dt.predict(test)
dt_predict_time = time.time() - start
mlp = MLPClassifier(hidden_layer_sizes=(100, 100), max_iter=1000, random_sta
start = time.time()
mlp.fit(train, target)
mlp_train_time = time.time() - start
```

```
start = time.time()
mlp.predict(test)
mlp_predict_time = time.time() - start
print(f"Decision Tree - Training: {dt_train_time}s, Prediction: {dt_predict_
print(f"MLP - Training: {mlp_train_time}s, Prediction: {mlp_predict_time}s")
```

```
Decision Tree - Training: 0.008005380630493164s, Prediction: 0.0007219314575
195312s
MLP - Training: 2.271789789199829s, Prediction: 0.0008683204650878906s
```

From the above result, the Decision Tree is faster for both training and predictions operations

[2.5 pts] Compare and contrast Decision Trees and MLPs.

Describe at least one advantage and disadvantage of using an MLP over a Decision Tree.

An advantage of MLP is that an MLP can learn non-linear relationships between features which can't be captured by a Decision Tree. A disadvantage of using an MLP is that they lack interpretability and is harder to understand rules and relationships compared to Decision Trees.

(15 pts) PCA

[2.5 pts] Transform the train data using PCA

Train a PCA model to project the train data on the top 8 components. **Print out the 8 principal components**. Look at the documentation of PCA for reference.

In [25]:
```
pca = PCA(n_components=8, random_state=SEED)
train_pca = pca.fit_transform(train)
print(pca.components_)
```

```
[[-5.88824889e-02 -3.60497119e-02 -1.70206867e-02  1.03789323e-01
  -1.12949393e-01  1.34058041e-01 -1.34058041e-01 -3.43838552e-01
   1.49148921e-01  1.90743643e-01  3.94598830e-03  1.77659035e-04
  -1.77659035e-04 -1.62590021e-01  1.64229135e-01 -1.63911390e-03
   3.49296677e-01 -3.49296677e-01 -4.42573564e-02 -2.98125292e-01
   3.42382648e-01  2.09179099e-01 -8.88114413e-02 -7.37523677e-02
  -6.43314353e-02  1.77161452e-02  2.71173534e-04 -4.76811068e-02
   3.30445838e-01 -2.83035904e-01]
 [-6.13278432e-02 -3.30647171e-02 -3.82416019e-02  1.28970449e-02
   6.12059254e-04 -4.16087229e-01  4.16087229e-01 -6.75333095e-02
   2.01325731e-02  4.83244503e-02 -9.23713979e-04  5.36809382e-02
  -5.36809382e-02 -4.76354860e-01  4.90696705e-01 -1.43418452e-02
  -1.66001158e-02  1.66001158e-02  1.61815136e-02 -1.26976371e-01
   1.10794857e-01  1.37704407e-02  1.88951894e-02 -5.13584946e-02
  -9.62605522e-03  2.83189197e-02 -2.43409967e-03  1.79759764e-03
  -2.57549131e-01  2.58185633e-01]
 [ 4.30373267e-02  3.36559313e-02 -6.49260472e-03  4.73583935e-02
  -1.94009792e-02 -3.32818332e-01  3.32818332e-01 -5.82053006e-02
  -2.06697991e-02  5.39898572e-02  2.48852425e-02 -1.88642130e-01
   1.88642130e-01  3.83618074e-01 -3.61087011e-01 -2.25310624e-02
   9.47378849e-02 -9.47378849e-02  2.93911226e-02 -3.73356525e-01
   3.43965403e-01 -3.06902783e-01  1.90250785e-01  6.09051989e-02
   4.51949662e-02  1.05518330e-02 -4.18105418e-03  1.36172655e-02
  -4.48751539e-02  3.54389426e-02]
 [ 1.72735405e-02 -5.28825152e-02 -9.96363275e-03 -4.71841404e-02
  -3.21543406e-02  1.28392814e-01 -1.28392814e-01  5.91001837e-02
  -3.40025736e-02  9.60286041e-02 -1.21126214e-01 -2.39879972e-02
   2.39879972e-02 -1.99845870e-01  2.02181018e-01 -2.33514831e-03
  -3.89234783e-01  3.89234783e-01 -3.23208400e-02 -1.67937182e-01
   2.00258022e-01 -3.90108929e-01  4.04300020e-01  1.03541209e-02
  -3.73497862e-02  1.28045745e-02 -2.56300197e-03 -5.58848629e-02
   3.08111509e-01 -2.49663644e-01]
 [ 7.47627151e-02  2.48266082e-02  4.04749115e-03 -4.42984136e-02
   1.99662995e-02  6.95875508e-02 -6.95875508e-02 -4.04735437e-01
  -7.10208393e-02  4.09316190e-01  6.64400859e-02 -2.15572063e-01
   2.15572063e-01 -8.05245268e-02  7.81944006e-02  2.33012622e-03
   1.94517362e-01 -1.94517362e-01 -5.96745297e-02  3.60779306e-01
  -3.01104776e-01 -3.66263814e-01  2.78404074e-01  3.95766679e-02
   1.83366074e-02  2.99464654e-02  7.32923906e-04 -5.92763654e-03
  -9.94055621e-02  1.04600275e-01]
 [ 4.40854366e-02  2.46721515e-02  3.67183124e-02  5.75119919e-03
   3.70473825e-02  3.79442099e-01 -3.79442099e-01  9.34862262e-02
   6.98051771e-03 -3.08609533e-02 -6.96057906e-02  1.49993996e-01
  -1.49993996e-01 -3.55499594e-02  4.19664572e-02 -6.41649785e-03
   1.17826587e-01 -1.17826587e-01  3.62750243e-02 -2.90934176e-01
   2.54659152e-01 -2.37672406e-01  4.70478898e-02  1.15540681e-01
   7.94044922e-02 -4.32065690e-03 -5.13010827e-03 -1.03489851e-01
  -3.76100441e-01  4.84720401e-01]
 [ 1.50904843e-02 -8.26562026e-02 -6.96669208e-03 -1.59665186e-02
  -2.42578740e-02 -1.30401050e-01  1.30401050e-01  2.21396778e-01
   4.35730066e-02 -2.08860049e-01 -5.61097361e-02  4.27838082e-01
  -4.27838082e-01 -6.81235342e-03 -1.17700683e-03  7.98936025e-03
   3.11279827e-01 -3.11279827e-01 -6.86037070e-02  1.69803378e-01
  -1.01199671e-01 -3.47670519e-01  2.64421539e-01  3.72261070e-02
   1.51708269e-02  3.08520464e-02 -3.09748886e-02  4.13864056e-02
   1.68707902e-01 -1.79119419e-01]
```

```
[-2.97311833e-02 -3.61033214e-02  2.30771943e-02  9.03087358e-03
 -6.66062121e-03 -5.09894872e-02  5.09894872e-02 -2.92850765e-01
 -1.89291973e-01  4.90145517e-01 -8.00277937e-03  4.10522690e-01
 -4.10522690e-01  2.01792583e-01 -2.01558281e-01 -2.34301568e-04
 -1.69611142e-01  1.69611142e-01  4.21749517e-03 -1.80916827e-02
  1.38741875e-02  1.99110630e-01  1.15018374e-01 -2.46470035e-01
 -7.89518397e-02  1.12928705e-02 -2.40972758e-03 -1.21356651e-01
 -1.29577769e-02  1.36724155e-01]]
```

[2.5 pts] Percentage of variance explained by top 8 principal components

Using PCA's "explained_variance_ratio_", print the percentage of variance explained by the top 8 principal components.

In [26]:
```
print(pca.explained_variance_ratio_ * 100)
```

```
[23.40265324 13.97151557 10.08586045  8.11110555  7.50913132  6.75425406
  5.99149126  4.95133091]
```

[2.5 pts] Transform the train and test data into train_pca and test_pca using PCA

Note: Use fit_transform for train and transform for test

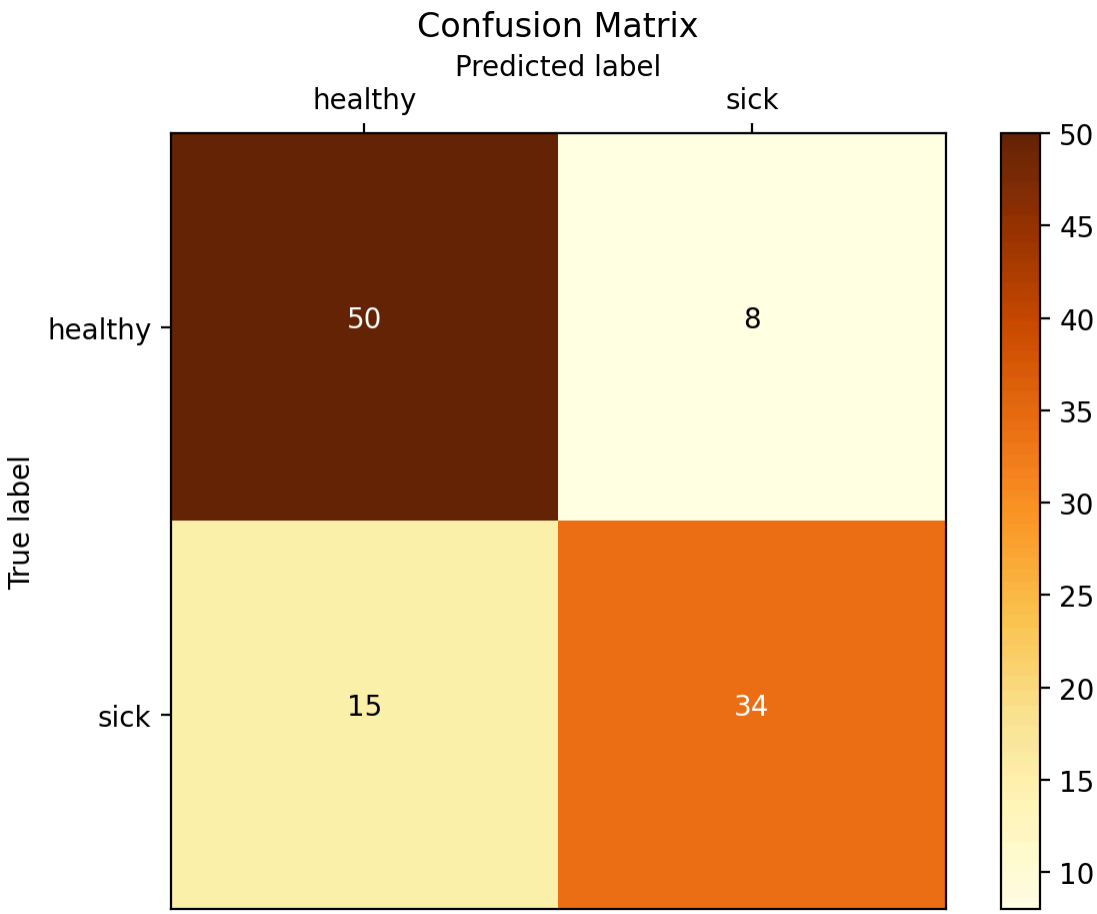
In [27]:
```
train_pca = pca.fit_transform(train)
test_pca = pca.transform(test)
```

[2.5 pts] PCA+Decision Tree

Train the default Decision Tree Classifier using train_pca. **Report the accuracy using test_pca and print the confusion matrix**.

In [30]:
```
dt_pca = DecisionTreeClassifier(random_state=SEED)
dt_pca.fit(train_pca, target)
test_predictions_pca = dt_pca.predict(test_pca)
test_accuracy_pca = metrics.accuracy_score(target_test, test_predictions_pca
print("%-12s %f" % ("Accuracy:", test_accuracy_pca))
print("Confusion Matrix: \n", metrics.confusion_matrix(target_test, test_pre
draw_confusion_matrix(target_test, test_predictions_pca, ["healthy", "sick"]
```

```
Accuracy:    0.785047
Confusion Matrix:
 [[50  8]
 [15 34]]
```



Does the model perform better with or without PCA? Give an explanation

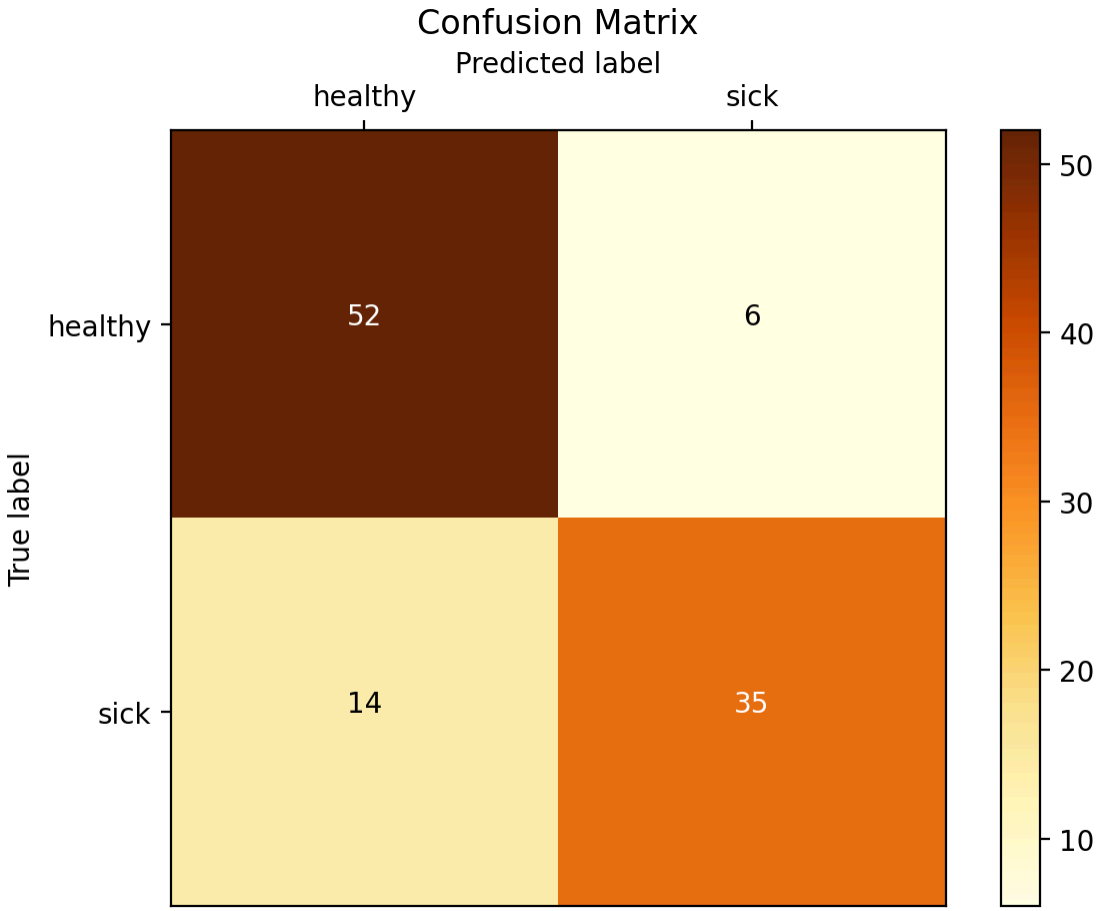
The model performs equally as well with or with PCA. This means the top 8 components captured the necessary information for classification. Furthermore, the reduced dimensionality from PCA shows that PCA removed redundant features successfully, giving us the same result of 78.5%.

[2.5 pts] PCA+MLP

Train the MLP classifier with the same parameters as before using train_pca. **Report the accuracy using test_pca and print the confusion matrix**.

In [33]:
```
mlp_pca = MLPClassifier(hidden_layer_sizes=(100, 100), max_iter=1000, random
mlp_pca.fit(train_pca, target)
test_predictions_mlp_pca = mlp_pca.predict(test_pca)
test_accuracy_mlp_pca = metrics.accuracy_score(target_test, test_predictions
print("%-12s %f" % ("Accuracy:", test_accuracy_mlp_pca))
print("Confusion Matrix: \n", metrics.confusion_matrix(target_test, test_pre
draw_confusion_matrix(target_test, test_predictions_mlp_pca, ["healthy", "si
```

```
Accuracy:    0.813084
Confusion Matrix:
 [[52  6]
 [14 35]]
```



Does the model perform better with or without PCA? Give an explanation

The model performed better without PCA as it performed about 2% better. MLP is great for capturing non-linear relationships. Therefore when we reduced dimensionality with PCA, some of those relationships were lost and therefore accuracy decreased.

[2.5 pts] Pros and Cons of PCA

In your own words, provide at least two pros and at least two cons for using PCA

An advantage of using PCA is that due to the reduction of features, we reduce computional resources used which makes our algorithm faster and more efficient. Another advantage is that PCA tries to maximize variance which helps remove noise and helps with generalization. A disadvantage of PCA is that we are only left with principle components that are linear combinations, leaving us

little interpretability and makes it hard to understand. Another disadvantage comes from dimensionality reduction which could lose some of the non-linear relationships we had captured previously.

(10 pts) K-Means Clustering

[2.5 pts] Apply K-means to the train data and print out the Inertia score

Use n_cluster = 15 and `random_state = SEED`.

In [35]:
```
kmeans = KMeans(n_clusters=15, random_state=SEED)
kmeans.fit(train)
print(kmeans.inertia_)
```

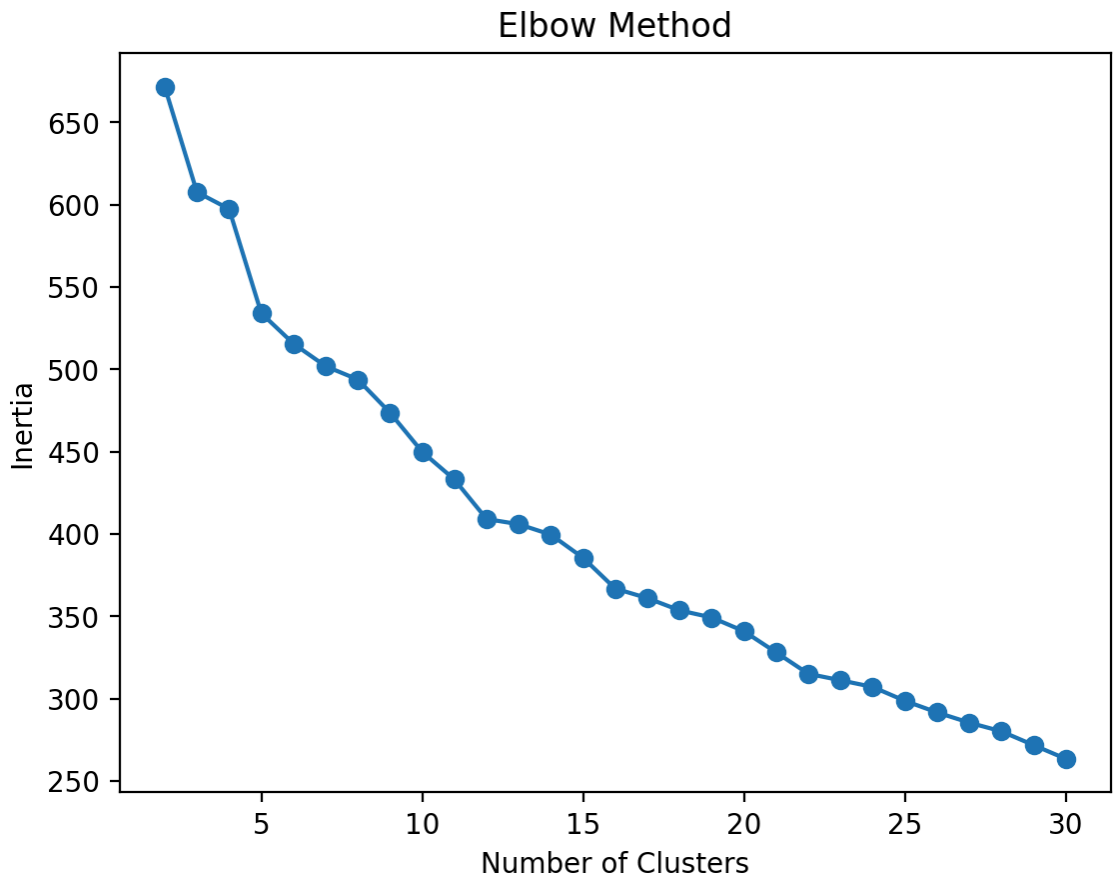
```
385.5118190986343
```

[5 pts] Find the optimal cluster size using the elbow method.

Use the elbow method to find the best cluster size or range of best cluster sizes for the train data. Check the cluster sizes from 2 to 30. Make sure to plot the Inertia and state where you think the elbow starts. Make sure to use `random_state = SEED`.

In [37]:
```
inertias = []
for k in range(2, 31):
    kmeans = KMeans(n_clusters=k, random_state=SEED)
    kmeans.fit(train)
    inertias.append(kmeans.inertia_)

plt.plot(range(2, 31), inertias, 'o-')
plt.xlabel('Number of Clusters')
plt.ylabel('Inertia')
plt.title('Elbow Method')
plt.show()
```



What seems to be the best range of cluster sizes?

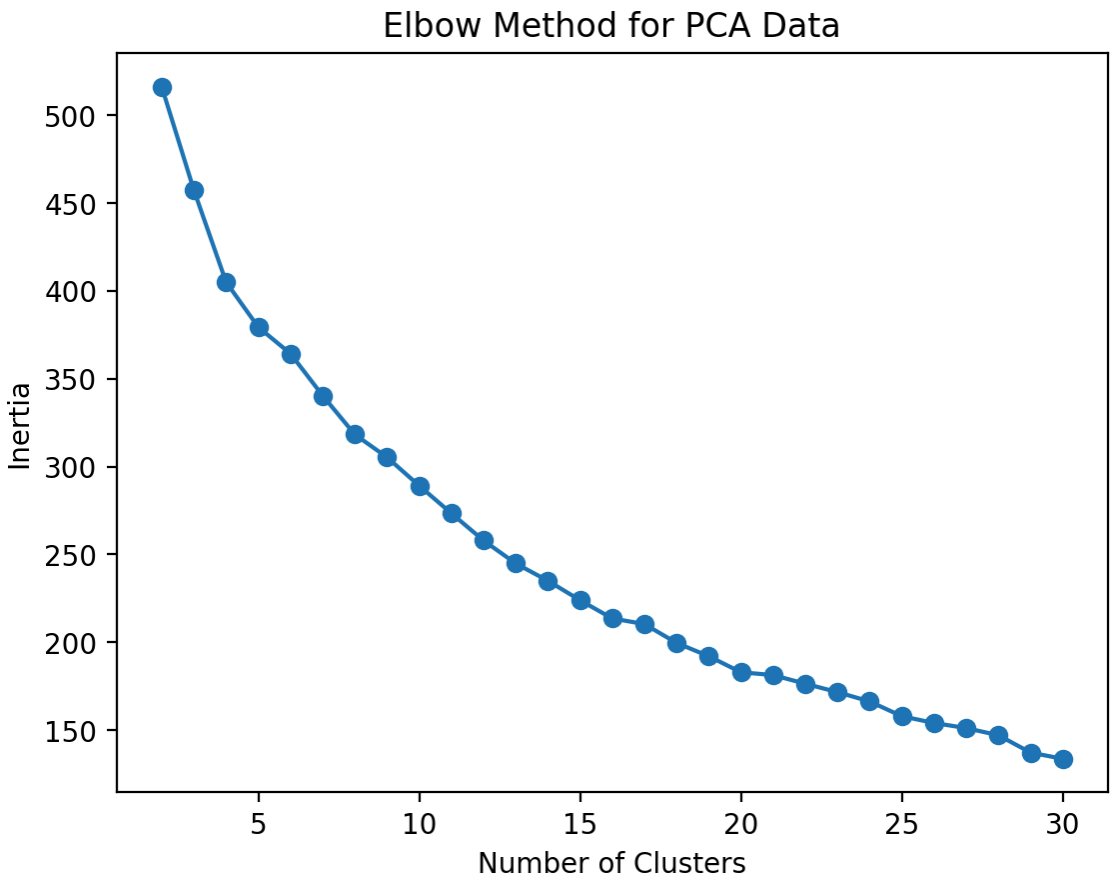
The best range seems to be from k = 11 to k = 14 since the curve starts to flatten more here.

[2.5 pts] Find the optimal cluster size for the train_pca data

Repeat the same experiment but use train_pca instead of train.

In [39]:
```
inertias_pca = []
for k in range(2, 31):
    kmeans_pca = KMeans(n_clusters=k, random_state=SEED)
    kmeans_pca.fit(train_pca)
    inertias_pca.append(kmeans_pca.inertia_)

plt.plot(range(2, 31), inertias_pca, 'o-')
plt.xlabel('Number of Clusters')
plt.ylabel('Inertia')
plt.title('Elbow Method for PCA Data')
plt.show()
```



What seems to be the best range of cluster sizes? Explain as well any differences between the resulting curve and the one from the previous exercise.

The best range of cluster sizes here seem to appear from k = 7 to k = 11. Due to the reduced dimensionality from PCA, the inertia values are smaller. Furthermore, the curve is smoother compared to the non-PCA graph. The PCA curve also flattens earlier so it shows that reducing dimensionality can make it easier to have good clustering with fewer clusters.

This notebook was converted with convert.ploomber.io

Project 3b

This is the second part of the project where will ask you to perform your own data science project to classify a new dataset.

Submission Details

Project is due June 13th at 11:59 pm (Friday Midnight). To submit the project, please save the notebook as a pdf file and submit the assignment via Gradescope. In addition, make sure that all figures are legible and sufficiently large. For best pdf results, we recommend printing the notebook using LaTeX

Loading Essentials and Helper Functions

In [1]:
```
# fix for windows memory leak with MKL
import os
import platform

if platform.system() == "Windows":
    os.environ["OMP_NUM_THREADS"] = "2"
```

In [2]:
```
# import libraries
import time
import random
import numpy as np  # linear algebra
import pandas as pd  # data processing, CSV file I/O (e.g. pd.read_csv)
import matplotlib.pyplot as plt  # this is used for the plot the graph

# Sklearn classes
from sklearn.model_selection import (
    train_test_split,
    cross_val_score,
    GridSearchCV,
    KFold,
)
from sklearn import metrics
from sklearn.metrics import confusion_matrix, silhouette_score
import sklearn.metrics.cluster as smc
from sklearn.cluster import KMeans
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.pipeline import Pipeline, FeatureUnion
from sklearn.preprocessing import (
    StandardScaler,
    OneHotEncoder,
    LabelEncoder,
    MinMaxScaler,
```

```
)
from sklearn.compose import ColumnTransformer, make_column_transformer
from sklearn import tree
from sklearn import datasets
from sklearn.decomposition import PCA
from sklearn.neural_network import MLPClassifier
from sklearn.datasets import make_blobs

from helper import (
    draw_confusion_matrix,
    heatmap,
    make_meshgrid,
    plot_contours,
    draw_contour,
)

%matplotlib inline
%config InlineBackend.figure_format = 'retina'

# Sets random seed for reproducibility
SEED = 66
random.seed(SEED)
```

In [3]:

```
from sklearn.metrics import f1_score

#If you encode the target labels with a different format (i.e. binary number
def f1(y_true,y_predicted):
    #y_true: Ground truth (correct) target values
    #y_predicted: Estimated targets as returned by a classifier.
    return f1_score(y_true, y_predicted, pos_label="Yes")

from sklearn.metrics import f1_score
from sklearn.metrics import make_scorer

f1_scorer = make_scorer(f1_score, pos_label="Yes") #Make use of this one for
```

(50 pts) Putting it all together: Classify your own data

Through the course of this program, you have acquired knowledge and skills in applying various models to tackle supervised learning tasks. Now, we challenge you to harness your cumulative learning and create a model capable of predicting whether it will rain tomorrow according to the current weather conditions.

Context

Predicting whether it will rain tomorrow is vital for daily decision-making in sectors like agriculture, transportation, and event planning, helping to reduce risks and optimize operations. It also plays a crucial role in public safety and

resource management, allowing individuals and authorities to prepare for potential weather disruptions.

Challenge

The goal of this project is to develop a predictive model that can determine whether it will rain tomorrow according to the available input parameters.

While we will provide specific instructions to guide you in the right direction, you have the freedom to choose the models and preprocessing techniques that you deem most appropriate. Upon completion, we request that you provide a detailed description outlining the models you selected and the rationale behind your choices.

Data Description

The dataset includes 23 columns. The information provided by each column is as follows:

- **Date:** The date of observation
- **Location:** The common name of the location of the weather station
- **MinTemp:** The minimum temperature in degrees celsius
- **MaxTemp:** The maximum temperature in degrees celsius
- **Rainfall:** The amount of rainfall recorded for the day in mm
- **Evaporation:** The so-called Class A pan evaporation (mm) in the 24 hours to 9am
- **Sunshine:** The number of hours of bright sunshine in the day.
- **WindGustDir:** The direction of the strongest wind gust in the 24 hours to midnight
- **WindGustSpeed:** The speed (km/h) of the strongest wind gust in the 24 hours to midnight
- **WindDir9am:** Direction of the wind at 9am
- **WindDir3pm:** Direction of the wind at 3pm
- **WindSpeed9am:** Wind speed (km/hr) averaged over 10 minutes prior to 9am
- **WindSpeed3pm:** Wind speed (km/hr) averaged over 10 minutes prior to 3pm
- **Humidity9am:** Humidity (percent) at 9am
- **Humidity3pm:** Humidity (percent) at 3pm
- **Pressure9am:** Atmospheric pressure (hpa) reduced to mean sea level at 9am
- **Pressure3pm:** Atmospheric pressure (hpa) reduced to mean sea level at 3pm

- **Cloud9am:** Fraction of sky obscured by cloud at 9am. This is measured in "oktas", which are a unit of eigths. It records how many eigths of the sky are obscured by cloud. A 0 measure indicates completely clear sky whilst an 8 indicates that it is completely overcast.
- **Cloud3pm:** Fraction of sky obscured by cloud (in "oktas": eighths) at 3pm. See Cload9am for a description of the values
- **Temp9am:** Temperature (degrees C) at 9am
- **Temp3pm:** Temperature (degrees C) at 3pm
- **RainToday:** Boolean: 1 if precipitation (mm) in the 24 hours to 9am exceeds 1mm, otherwise 0
- **RainTomorrow:** The amount of next day rain in mm. Used to create response variable RainTomorrow. A kind of measure of the "risk".

(25 pts) Preprocessing

For the dataset, the following are mandatory pre-processing steps for your data:

- **Use One-Hot Encoding on categorical features if needed** (specify whether you keep the extra feature or not for features with multiple values)
- **Determine which fields if any need to be dropped**
- **Handle missing values** (Specify your strategy)
- **Rescale the real valued features using any strategy you choose** (StandardScaler, MinMaxScaler, Normalizer, etc)
- **Augment at least one feature**
- **Implement a strategy to balance your dataset according to the target class**
- **Implement a train-test split with 20% of the data going to the test data**. Make sure that the test and train data are balanced in terms of the desired class.

After writing your preprocessing code, write out a description of what you did for each step and provide a justification for your choices. All descriptions should be written in the markdown cells of the jupyter notebook. Make sure your writing is clear and professional.

We highly recommend reading through the scikit-learn documentation to make this part easier.

The dataset is found in **datasets/weather_data.csv**

In [27]:
```
from sklearn.utils import resample

data = pd.read_csv("datasets/weather_data.csv")
#Drop necessary fields
data_clean = data.dropna(subset=['RainTomorrow']).drop(columns=['Date', 'Rai
```

```
target = data.dropna(subset=['RainTomorrow'])['RainTomorrow']
numerical_cols = data_clean.select_dtypes(include=[np.number]).columns.tolis
categorical_cols = data_clean.select_dtypes(include=['object']).columns.toli
#Handle missing values by using median
for col in numerical_cols:
    if data_clean[col].isnull().sum() > 0:
        data_clean[col] = data_clean[col].fillna(data_clean[col].median())
#Handle missing values by using mode
for col in categorical_cols:
    if data_clean[col].isnull().sum() > 0:
        data_clean[col] = data_clean[col].fillna(data_clean[col].mode()[0])
#Feature augmentation
data_clean['TempRange'] = data_clean['MaxTemp'] - data_clean['MinTemp']
data_clean['PressureChange'] = data_clean['Pressure3pm'] - data_clean['Press
data_clean['HumidityChange'] = data_clean['Humidity3pm'] - data_clean['Humic
categorical_features = ['Location', 'WindGustDir', 'WindDir9am', 'WindDir3pm
numerical_features = [col for col in data_clean.columns if col not in categc
#Train-test split with 20%
X_train, X_test, y_train, y_test = train_test_split(data_clean, target, test
#Rescale features
numerical_pipeline = Pipeline([('scaler', StandardScaler())])
#One hot encoding, decided to drop the extra feature to avoid collinearity
categorical_pipeline = Pipeline([('onehot', OneHotEncoder(drop='first', spar
preprocessor = ColumnTransformer([('num', numerical_pipeline, numerical_feat
X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)
feature_names = (numerical_features + list(preprocessor.named_transformers_[
train_df = pd.DataFrame(X_train_processed, columns=feature_names)
train_df['target'] = y_train.values
majority_class = train_df[train_df.target == 'No']
minority_class = train_df[train_df.target == 'Yes']
#Balance class
majority_undersampled = resample(majority_class, replace=False, n_samples=le
balanced_df = pd.concat([majority_undersampled, minority_class])
X_train_balanced = balanced_df.drop('target', axis=1).values
y_train_balanced = balanced_df['target'].values
```

First, I dropped the 'date' field since its data isn't appropriate for our prediction. The 'RainTomorrow' field was also dropped and used as the target variable. Then, I removed rows that have missing target values since they won't help us. For numerical features, I decided to fill the missing values with the median value since it will not be affected by outliers and won't skew the data. For categorical features, I decided to fill missing values with the mode since it naturally helps model categorical data that occurs frequently. Then I created new derived features which include 'TempChange', 'PressureChange', and 'HumidityChange'. A decrease in pressure can imply potential rain while steady pressure could imply normal weather. Lastly, the change in humidity can offer insight to moisture patterns to help us predict rainfall. Next, we split the dataset to 80% training and 20% testing as specified. Next, I applied the StandardScalar to all numerical features so all of these features will equally contribute to optimization. After, we applied one-hot encoding to categorical features and removed a

dummy variable from each feature. As a result, we can store data for our categorical features while also prohibiting multicollinearity. After processing and transforming the data, I sampled using stratified sampling so the training and testing sets will have the same ratio of rainy and nonrainy days. Finally, undersampling was applied to balance the dataset which helps prevent bias.

(25 pts) Try out a few models

Now that you have pre-processed your data, you are ready to try out different models.

For this part of the project, we want you to experiment with all the different models demonstrated in the course to determine which one performs best on the dataset.

You must perform classification using at least 3 of the following models:

- Logistic Regression
- K-nearest neighbors
- SVM
- Decision Tree
- Multi-Layer Perceptron

Due to the size of the dataset, be careful which models you use and look at their documentation to see how you should tackle this size issue for each model.

For full credit, you must perform some hyperparameter optimization on your models of choice. You may find the following scikit-learn library on hyperparameter optimization useful.

For each model chosen, write a justification, which parameters you optimized, and which parameters you choose for your best model. While the previous part of the project asked you to pre-process the data in a specific manner, you may alter pre-processing step as you wish to adjust for your chosen classification models.

Be sure to evaluate your models' performance not only with accuracy but other scores such as the f1 score that provide a more comprehensive evaluation. We provide a version of it at the beginning, make sure to check it! Try to get a test score above 60%

In [26]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import accuracy_score
```

```
lr_param_grid = {'C': [0.1, 1.0, 10.0], 'penalty': ['l1', 'l2'], 'solver': [
lr_grid = GridSearchCV(LogisticRegression(random_state=SEED, max_iter=1000),
lr_grid.fit(X_train_balanced, y_train_balanced)
print(f"Best Logistic Regression Parameters: {lr_grid.best_params_}")
lr_predictions = lr_grid.predict(X_test_processed)
lr_acc = accuracy_score(y_test, lr_predictions)
lr_f1 = f1(y_test, lr_predictions)
print(f"Optimized Logistic Regression - Accuracy: {lr_acc}, F1: {lr_f1}")

dt_param_grid = {'max_depth': [5, 10, 20, None], 'min_samples_split': [10, 2
dt_grid = GridSearchCV(DecisionTreeClassifier(random_state=SEED), dt_param_g
dt_grid.fit(X_train_balanced, y_train_balanced)
print(f"Best Decision Tree Parameters: {dt_grid.best_params_}")
dt_predictions = dt_grid.predict(X_test_processed)
dt_acc = accuracy_score(y_test, dt_predictions)
dt_f1 = f1(y_test, dt_predictions)
print(f"Optimized Decision Tree - Accuracy: {dt_acc}, F1: {dt_f1}")

knn_param_grid = {'n_neighbors': [3, 5, 7, 9], 'weights': ['uniform', 'dista
knn_grid = GridSearchCV(KNeighborsClassifier(), knn_param_grid, cv=3, scorir
knn_grid.fit(X_train_balanced, y_train_balanced)
print(f"Best KNN Parameters: {knn_grid.best_params_}")
knn_predictions = knn_grid.predict(X_test_processed)
knn_acc = accuracy_score(y_test, knn_predictions)
knn_f1 = f1(y_test, knn_predictions)
print(f"Optimized KNN - Accuracy: {knn_acc}, F1: {knn_f1}")
```

```
Best Logistic Regression Parameters: {'C': 1.0, 'penalty': 'l1', 'solver':
'liblinear'}
Optimized Logistic Regression - Accuracy: 0.7926790674777594, F1: 0.62669368
11447386
Best Decision Tree Parameters: {'criterion': 'gini', 'max_depth': 10, 'min_s
amples_leaf': 16, 'min_samples_split': 50}
Optimized Decision Tree - Accuracy: 0.7793171349203558, F1: 0.60323681881400
94
Best KNN Parameters: {'metric': 'euclidean', 'n_neighbors': 9, 'weights': 'd
istance'}
Optimized KNN - Accuracy: 0.7955272688913112, F1: 0.6249113074888731
```

The first model I picked was Logistic Regression since it is great at handling binary classification such as predicting rain. Logistic Regression scales great with large datasets and gives interpretable coefficients that can help us look at the data. For parameters, we optimized 'C' by testing [0.1, 1.0, 10.0] and 'penalty' by testing ['l1', 'l2']. The best parameters resulted in {'C': 1.0, 'penalty': 'l1', 'solver': 'liblinear'} which is what I chose for a 62.7% F1 score. The second model I picked was Decision Trees since they are great at handling datasets that have both categorical and numerical data. Decision Trees also have very interpretable rules which helps model relationships between features. For parameters, I optimized 'max_depth' by testing [5, 10, 20, None], 'min_samples_split' by testing [10, 20, 30, 50], 'min_samples_leaf' by testing [2, 4, 8, 16], and 'criterion' by testing ['gini', 'entropy']. The best parameters resulted in {'criterion': 'gini', 'max_depth': 10, 'min_samples_leaf': 16, 'min_samples_split': 50} which is what I

chose for a 60.3% F1 score. The last model I picked was KNN since it performs well at capturing local patterns which are very prevalent when trying to predict weather. For parameters, I optimized 'n_neighbors' by testing [3, 5, 7, 9], 'weights' by testing ['uniform', 'distance'], and 'metric' by testing ['euclidean', 'manhattan']. The best parameters resulted in {'metric': 'euclidean', 'n_neighbors': 9, 'weights': 'distance'} which is what I chose for a 62.5% F1 score.

Extra Credit

We will award a bonus point for the project for every percentage point over 60% that you get on any of your models' f1 test score using the provided **f1** function.

This notebook was converted with convert.ploomber.io